

THE TAILWIND CATALOGUE

Not the lecture. The thing you keep open in the other window while you work.

WHAT THIS IS

The translation table, the three scales, the variants, the syntax — and every lecture slide the cycles replaced.

WHAT IT IS NOT

Not examined. It is optional depth; everything graded was typed in a cycle.

BETTER THAN THIS

`demos/03- translator/` answers by measuring.
Use it first.

THIS IS WHAT YOU SHIPPED LAST WEEK

[a notification card, and its stylesheet, measured](#) [↶ reload](#) [open ↷](#)

WRITTEN BY HAND — THE WAY WEEK 5 TAUGHT

! **Certificate expires in 6 days**
Renew it before Friday or the gateway starts refusing connections.

rules in the stylesheet	15
...matching nothing on this page	8
class names invented	11
rules that escalated	7
!important	1
worst specificity (a,b,c)	1,2,0

8 of 15 authored rules (53%) match nothing on this page.
The browser found them just now, by running `querySelectorAll` on every selector.

Dead:
`.notice--compact`
`.notice--compact .notice__icon`
`.notice--compact .notice__title`

THE SAME CARD, IN UTILITIES

! **Certificate expires in 6 days**
Renew it before Friday or the gateway starts refusing connections.

rules in the stylesheet	0 — there is no stylesheet
...matching nothing on this page	0
class names invented	0
rules that escalated	0
!important	0
highest specificity possible	0,1,0

► **HANDS-ON** [demos/01-why-utilities/](#) — left column only. Count how many of those rules still do something. **DO NOT SCROLL RIGHT YET.**

FOUR COSTS, AND YOU PAID ALL OF THEM

DEAD RULES

Over half the stylesheet matches nothing on the page. Nobody deleted them, and nobody was careless — you cannot prove a selector is unused by reading it.

INVENTED NAMES

Eleven of them.

`.notice__meta--compact` is not a design decision. It is bookkeeping you have to remember.

THE SPECIFICITY RATCHET

Three rules fighting over one colour. To change it you cannot write a rule — you have to write one that **wins**. And the next person has to beat yours.

And the fourth: two files, and your attention moving between them all day.

THE RATCHET, IN THE FILE YOU WROTE

```
1  /* January */  
2  .notice__title { color: #0f172a; }  
3  
4  /* February – the panel needed a darker one */  
5  .panel .notice__title { color: #0f172a; }  
6  
7  /* March – the alert panel needed red */  
8  #alerts .panel .notice__title { color: #7f1d1d; }
```

Every step is reasonable. The stylesheet is now un-editable and nobody did anything wrong.

AND NOW THE BILL

The markup is three times longer, and it reads worse.

ADMIT IT FIRST

Say the cost before anyone else does. It is true and it does not go away.

THE HONEST COSTS PAY IT

Honestly, with the arguments the people who dislike this system actually make.

CYCLE 5 FIXES IT

Not by shortening the markup. By having one of it.

WHAT IS ACTUALLY BEING CLAIMED

- What it is** A fixed set of single-purpose classes, generated from your own design tokens.
- Key trait** The styling lives where the markup lives, so a component is one thing in one place instead of two things in two files.
- Use it for** Anything you are building rather than theming. It is the wrong tool for a stylesheet you ship to other people's pages.
- Tips** It is not faster to type. It is cheaper to **delete** — and deleting is what you spend a codebase's life doing.

Nothing here is new CSS. Every class you will see today emits a declaration you already wrote, in a file you already deleted.

What changes is **where the decision lives** — and that is the only thing worth arguing about.

MISCONCEPTION 1

“SO I HAVE TO MEMORISE A FEW HUNDRED CLASS NAMES.”

No. You have to know CSS, and you already do.

Someone who memorises is stuck the first time they meet a class they have not seen — and that happens tomorrow.



MISCONCEPTION 2

“THIS IS JUST INLINE STYLES WITH EXTRA STEPS.”

It is a fair objection, it is the first thing everyone says, and it deserves a real answer rather than a slogan.

ASK BOTH OF THEM TO DO FOUR THINGS

a style attribute and a utility, side by side↶ reloadopen ↷

narrow the frame dark add an overriding class viewport is >= md (48rem)

style="..."
Deploy

utilities
Deploy

asked to...	style=""	utilities	measured right now
change on hover	no	yes	a style attribute has no :hover to hang a value on
change at a breakpoint	no	yes	a style attribute has no @media
change in dark mode	no	yes	same reason – no @media
be overridden by a later class	no	yes	the attribute is (1,0,0,0) – it beats every selector you can write
hold a value computed at run time	yes	no	THE one thing the attribute is genuinely for – utilities are a fixed set

right now:

```
style="..." button    background rgb(24, 45, 242) · padding 8px 16px
utility button        background rgb(24, 45, 242) · padding 8px 32px
```

and here is what those utility classes actually ARE. Read out of document.styleSheets – this is the browser answering, not a lookup table:

► **HANDS-ON** [demos/02-not-inline-styles/](#) — press the buttons in order: **HOVER · NARROW · DARK · OVERRIDE**. In each step, watch the left button.

BECAUSE A CLASS CAN CARRY A CONDITION. AN ATTRIBUTE CANNOT.

```
1 @media (hover: hover) {  
2   .hover\:bg-brand-700:hover { background-color: ... } }  
3  
4 @media (width >= 48rem) {  
5   .md\:px-8 { padding-inline: ... } }  
6  
7 @media (prefers-color-scheme: dark) {  
8   .dark\:bg-brand-300 { background-color: ... } }
```

An attribute is a value. These are conditions.

WHAT A STYLE ATTRIBUTE IS GENUINELY FOR

A VALUE COMPUTED AT RUN TIME

A progress bar at 63%. A tooltip at the cursor. A number that did not exist when the page was written. Utilities are a fixed set generated from the source.

THAT IS THE WHOLE LIST

And it is worth saying, because a rebuttal with a boundary is stronger than one without. You will use it in week 10.

AND IT NEVER FIGHTS THE CSS YOU DO WRITE

```
1 @layer theme, base, components, utilities; /* Tailwind, first thing */
2
3 @layer base { h2 { font-size: 1.5rem } } /* all of it: LAYERED */
4
5 h2 { font-size: 13px } /* yours is not – and it wins */
```

A layered rule loses to an unlayered one whatever the order. Your CSS beats the reset with no `!important`.

PREFLIGHT: THE RESET YOU ALREADY WROTE

WHAT THE SCRIPT LINE ALONE DOES	MEASURED	YOU MET IT IN
<code>box-sizing: border-box</code> on everything	the cycle-1 card: 418 to 384 wide	week 3's three lines
headings lose size and weight	<code>h1</code> : 32px bold, now 16px and 400	you give them back in classes
margins on headings and paragraphs: 0	<code>h1, p</code> : margin 0	—
a system font stack, line-height 1.5	Times becomes the system sans	week 3's <code>system-ui</code>
lists without bullets or indent	<code>ul</code> : list-style none, padding 0	week 4's nav list
images block, <code>max-width: 100%</code>	<code>img</code> : display block	week 5's images
buttons and links inherit	button background transparent, link colour inherited	week 3's <code>font: inherit</code>

All of it sits in `@layer base`. Your own unlayered CSS beats it whatever the order — which is why the cycle-1 card did not move until you deleted its rules.

WORKED EXAMPLE

ONE CARD, TRANSLATED IN FRONT OF YOU

- 1 Open last week's `.kpi` rule. Read it out loud, declaration by declaration.
- 2 Write the class for each one. Delete the CSS line you just replaced.
- 3 When the rule is empty, delete the rule. Then delete the file.
- 4 Reload. Nothing changed on screen.

Do not look anything up. Guess each class, get one wrong, and fix it.

ASK THE BROWSER INSTEAD

CSS in, utility out – by measurement, not by table

UTILITY → CSS

a class, exactly as you would write it in the markup

p-4

p-4md:px-8hover:bg-brand-700dark:bg-slate-900flexgap-4rounded-xltext-huge

```
.p-4 {  
  padding: calc(var(--spacing) * 4);  
}
```

...and that is all it is. One class, one rule.

applied to a real element, live:

specimen

CSS → UTILITY

a declaration, straight out of last week's stylesheet

padding: 1rem;

padding: 1rem; display: flex; gap: 1rem; justify-content: space-between;

font-weight: 600; padding: 13px; max-width: 62ch;

you asked for:
padding-top: 16px
padding-right: 16px
padding-bottom: 16px
padding-left: 16px

these classes resolve to exactly that:
p-4

Checked by MEASURING, not by looking up a table: each one was applied to a real element and its computed value compared with yours. That is why the answer is trustworthy even for a property nobody anticipated.

► **HANDS-ON** [demos/03-translator/](#) — start with the **RIGHT** box. `padding: 1rem`, then `display: flex`, then `justify-content: space-between`. Stop before the fourth.

THE WHOLE TRANSLATION, BY PROPERTY FAMILY

BOX

`padding` → `p-*` · `px-*` · `py-*` · `pt-*`
`margin` → `m-*` · `mx-auto`
`border-radius` → `rounded-*`

LAYOUT

`display:flex` → `flex`
`grid-template-columns` → `grid-cols-*`
`gap` → `gap-*` · `gap-x-*`

ALIGNMENT

`justify-content` → `justify-*`
`align-items` → `items-*`
`align-self` → `self-*`

TYPE

`font-size` → `text-sm` · `text-lg`
`font-weight` → `font-semibold`
`text-decoration` → `underline` ·
`no-underline`

COLOUR

`color` → `text-*`
`background` → `bg-*`
`border-color` → `border-*`

THE ONE TRAP

`text-*` means **both** size and colour. `text-sm` is a size; `text-slate-600` is a colour. The value decides.

BOX — SPACING, SIZE, BORDER

CSS YOU WROTE	UTILITY	NOTE
<code>padding: 1rem</code>	<code>p-4</code>	4 × the 0.25rem step
<code>padding-inline: 1.5rem</code>	<code>px-6</code>	<code>py-*</code> for block
<code>padding-top: .5rem</code>	<code>pt-2</code>	<code>pt</code> , <code>pr</code> , <code>pb</code> , <code>pl</code>
<code>margin: 0</code>	<code>m-0</code>	<code>-mt-4</code> is negative
<code>margin-inline: auto</code>	<code>mx-auto</code>	the centring one
<code>margin-block-start: auto</code>	<code>mt-auto</code>	pushes to the bottom of a flex column
<code>width: 100%</code>	<code>w-full</code>	<code>w-screen</code> , <code>w-fit</code> , <code>w-4</code>
<code>max-width: 36rem</code>	<code>max-w-xl</code>	named steps, not the spacing scale
<code>border: 1px solid</code>	<code>border</code>	<code>border-2</code> , <code>border-t</code>
<code>border-radius: .75rem</code>	<code>rounded-xl</code>	<code>rounded-full</code> for a pill
<code>box-sizing: border-box</code>	—	already the default. It is in the reset.

LAYOUT — FLEX AND GRID

CSS YOU WROTE	UTILITY	NOTE
<code>display: flex</code>	<code>flex</code>	<code>inline-flex</code> , <code>grid</code> , <code>block</code> , <code>hidden</code>
<code>flex-direction: column</code>	<code>flex-col</code>	<code>flex-row</code> is the default
<code>flex-wrap: wrap</code>	<code>flex-wrap</code>	grid never needs it
<code>flex: 1 (= 1 1 0%)</code>	<code>flex-1</code>	equal shares, whatever the text. <code>flex-auto</code> is <code>1 1 auto</code>
<code>flex-basis: 0</code>	<code>basis-0</code>	after <code>flex-1</code> it adds nothing — measured
<code>gap: 1rem</code>	<code>gap-4</code>	<code>gap-x-*</code> , <code>gap-y-*</code>
<code>grid-template-columns: repeat(3, 1fr)</code>	<code>grid-cols-3</code>	up to <code>grid-cols-12</code>
<code>grid-column: span 2</code>	<code>col-span-2</code>	<code>col-start-*</code> , <code>col-end-*</code>
<code>grid-template-areas</code>	no utility	use the 12-column form, or brackets
<code>min-width: 0</code>	<code>min-w-0</code>	week 4's overflow fix, unchanged
<code>position: sticky</code>	<code>sticky</code>	with <code>top-0</code>

flex-1 IS ALREADY A ZERO BASIS

ON EACH OF THREE TILES	COMPUTED FLEX	WIDTHS AT 1280 (CYCLE 2)
nothing (inside flex)	0 1 auto	69 · 473 · 73 — each as wide as its text wants
flex-auto	1 1 auto	69 · 473 · 73 — the free space shared, the text first
flex-1	1 1 0%	205 · 205 · 205
flex-1 basis-0	1 1 0px	205 · 205 · 205 — no change

WHY

Tailwind 4 compiles `flex-1` to `flex: 1`, and the CSS shorthand with one number sets the basis to `0%`. The grow factor then shares **all** of the row.

WHAT TO WRITE ABOUT IT

A class that changes nothing is still a class somebody has to read. In a review, "`basis-0` is redundant after `flex-1`" is a sentence about the code — with a measurement behind it.

TYPE AND COLOUR

CSS	UTILITY
<code>font-size: .875rem</code>	<code>text-sm</code>
<code>font-size: 1.5rem</code>	<code>text-2xl</code>
<code>font-weight: 600</code>	<code>font-semibold</code>
<code>line-height: 1.25</code>	<code>leading-tight</code>
<code>text-align: center</code>	<code>text-center</code>
<code>text-decoration: none</code>	<code>no-underline</code>
<code>text-transform: uppercase</code>	<code>uppercase</code>
<code>font-variant-numeric</code>	<code>tabular-nums</code>

CSS	UTILITY
<code>color: ...</code>	<code>text-brand-500</code>
<code>background: ...</code>	<code>bg-surface</code>
<code>border-color: ...</code>	<code>border-edge</code>
<code>outline-color: ...</code>	<code>outline-brand-500</code>
<code>opacity on a colour</code>	<code>bg-white/15</code>
<code>opacity: .5</code>	<code>opacity-50</code>

text-* is both. **text-sm** is a size, **text-slate-600** is a colour. The value decides.

THE SCALES ARE THE PRODUCT. THE SHORT NAMES ARE A SIDE EFFECT.

spacing, type and colour, drawn at their real sizes

↻ reload

open ↗

SPACING — EVERY STEP IS --SPACING TIMES A WHOLE NUMBER

w-0.5	2px	
w-1	4px	
w-1.5	6px	
w-2	8px	
w-3	12px	
w-[13px]	13px	
w-4	16px	
w-5	20px	
w-6	24px	
w-8	32px	
w-10	40px	
w-12	48px	
w-16	64px	
w-20	80px	
w-24	96px	

TYPE

text-xs	12px	Renew the certificate
text-sm	14px	Renew the certificate
text-base	16px	Renew the certificate

► **HANDS-ON** [demos/04-scale/](#) — cover the readout and decide: **WHICH ONE DOES NOT BELONG?**

THE THREE SCALES

SPACING — 0.25rem PER STEP

0 · 0.5 · 1 · 1.5 · 2 · 3 · 4 ·

Seeing a number in old CSS? **Divide by four.** 16px is 4.

TYPE — NAMED, NOT NUMBERED

xs · sm · base · lg · xl · 2xl ·

Each carries a line-height with it.
Override with `leading-*`.

COLOUR — 50 → 950

brand-100 · brand-300 ·
brand-500 · brand-700

Higher is darker. **Which steps are legal for text is a contrast question**, not a palette one.

All three come out of `@theme`. Add a name there and its whole family of utilities exists.

ONE NUMBER MOVES THE WHOLE PAGE

```
1 @theme {  
2   --spacing: 0.25rem;  
3 }
```

`p-4` is not “sixteen pixels”. It is **four units**, and the unit is one line.

What it is

A single multiplier behind every padding, margin, gap, width and height.

Key trait

Change it and everything moves together, staying in proportion.

Use it for

Denser layouts, a bigger touch target, a design that felt cramped.

Tip

This is week 3's token argument with a system on top. Same idea, more reach.

@theme — THE SYNTAX

```
1 <style type="text/tailwindcss"> <!-- the type attribute is what marks it -->
2   @theme {
3     /* a colour: gives you text-*, bg-*, border-*, ring-*, outline-* */
4     --color-brand-500: hsl(234 89% 52%);
5
6     /* the spacing multiplier: one number behind p-*, m-*, gap-*, w-*, h-* */
7     --spacing: 0.25rem;
8   }
9 </style>
```

IT IS ALSO A CUSTOM PROPERTY

`var(--color-brand-500)` works everywhere it worked in week 3. Nothing was replaced.

THE NAMING IS THE API

`--color-*` makes colour utilities.
`--text-*` makes sizes,
`--radius-*` makes corners. The prefix decides the family.

NO BUILD STEP

Which is also why the block is repeated on every page — the price of loading Tailwind from one file with no build.

MISCONCEPTION 4

SQUARE BRACKETS: A TEST, NOT A TABOO

three legitimate uses and one smell, audited

↶ reload

open ↷

The test, and it is mechanical:

Would you want a *second* element to have this exact value?

yes → it is a decision. Give it a name in @theme.

no → it is a one-off. Brackets are the right answer.

And then: is it already on the scale, or within one step of it? If so you are overriding a system that was working, and that is the smell.

CORRECT A SIZE SOMEBODY ELSE DECIDED

A leaderboard slot is 728x90 because the ad network says so. Nobody on this team chose it, nobody will reuse it, and rounding it to the scale would break the thing. Brackets, no apology.

728 × 90

```
class="w-[728px] h-[90px] ..."
```

measured 0 × 0 – exactly the number the ad network specified.

Nobody would want a *SECOND* element at 728x90, so there is nothing to name.

The test says brackets, and the test is right.

► **HANDS-ON** demos/08-arbitrary/ — third panel first. Ask yourself: **HOW MANY COLOURS ARE IN THIS ROW?**

ARBITRARY VALUES – SYNTAX, AND THE TEST

FORM	EXAMPLE	WHEN
arbitrary value	<code>w-[728px]</code>	a size somebody else decided
arbitrary property	<code>[mask-image:linear-gradient(...)]</code>	a property with no utility at all
underscores for spaces	<code>grid-cols-[repeat(auto-fit, minmax(11rem,1fr))]</code>	a pattern a utility cannot take
a theme value	<code>bg(--color-brand-500)</code>	rarely — the named class is better

“WOULD A SECOND ELEMENT WANT THIS EXACT VALUE?”

No → brackets are correct. Nobody chose it and nobody will reuse it.

YES

It is a decision. Put it in `@theme` and it gets a name.

ALREADY ON THE SCALE, OR ONE STEP OFF

This is the smell. `p-3` exists; `p-[13px]` is you losing.

WEEK 5'S RULE, NOW IMPOSSIBLE TO GET BACKWARDS

which breakpoints match, at any width you like

reload open

frame width 1400px sm md lg xl 2xl

Requests
1.42M

Latency
86ms

Errors
0.31%

WHICH BREAKPOINTS MATCH, RIGHT NOW

sm:	≥ 40rem (640px)	matches – not used in this markup
md:	≥ 48rem (768px)	matches
lg:	≥ 64rem (1024px)	matches – not used in this markup
	≥ 80rem	

► **HANDS-ON** [demos/05-responsive/](#) — start at 1400 and drag **DOWN**, slowly. Stop at exactly 768 and predict what happens at 767.

RESPONSIVE PREFIXES

PREFIX	GENERATES	ROUGHLY
(none)	no query at all	every width. Start here.
sm:	<code>@media (width >= 40rem)</code>	640px
md:	<code>@media (width >= 48rem)</code>	768px
lg:	<code>@media (width >= 64rem)</code>	1024px
xl:	<code>@media (width >= 80rem)</code>	1280px
2xl:	<code>@media (width >= 96rem)</code>	1536px
max-md:	<code>@media (width < 48rem)</code>	the escape hatch. Not a default.

THEY STACK, THEY DO NOT TAKE TURNS

At 1400px, `sm`, `md`, `lg` and `xl` all match. The last one that matches ends up on top. None of them fights.

THE BREAKPOINTS ARE IN `rem`

So a reader who enlarged their browser font gets them earlier — which is correct, and is why they are not written in pixels.

order MOVES THE PAINT, NOT THE TAB KEY

CYCLE 2, AT 375 — PAINTED TOP TO BOTTOM, AND THE TAB ORDER

```
<header class="order-first md:order-none"> y 16
<main class="order-first md:order-none"> y 94
<nav> y 540
Tab: menu links (y 560) first, Book early (y 169) last
```

What it is

`order-first` is

`order: -9999`: first of **every** item. Equal orders keep source order.

What it is not

A change to the document. Tab and a screen reader follow the DOM.

When it is fine

Neighbouring navigation and content. Never where the order **is** the meaning — a form, a sequence.

“You don't always need media queries — the system replaces most of them.” — *about a different framework. It was true then and it is true now.*

STATE VARIANTS — THE ONES YOU REACH FOR

VARIANT	GENERATES	NOTE
<code>hover:</code>	<code>@media (hover: hover) + :hover</code>	never sticks on a phone — free
<code>focus-visible:</code>	<code>:focus-visible</code>	the rubric line. Keyboard only
<code>focus:</code>	<code>:focus</code>	fires on mouse too. Usually you want the one above
<code>active:</code>	<code>:active</code>	only while the button is held down
<code>disabled:</code>	<code>:disabled</code>	a state, not a fifth variant
<code>dark:</code>	<code>@media (prefers-color-scheme: dark)</code>	the OS preference. No button
<code>motion-safe:</code>	<code>@media (prefers-reduced-motion: no-preference)</code>	put every transition behind it
<code>placeholder:</code>	<code>::placeholder</code>	pseudo-elements work too

READ THE SELECTOR AND THE MAGIC EVAPORATES

```
1 /* group-hover:text-brand-500 - inside hover: hover, like every hover variant */
2 @media (hover: hover) {
3   .group-hover\:text-brand-500:is(:where(.group):hover *) { ... }
4 }
5
6 /* peer-checked:text-brand-500 */
7 .peer-checked\:text-brand-500:is(:where(.peer):checked ~ *) { ... }
```

"INSIDE A HOVERED `.group`"

A descendant combinator and a pseudo-class. Week 3 taught both.

"A LATER SIBLING OF A CHECKED `.peer`"

The `~` combinator — which is why the checkbox must come **first** in the markup. CSS cannot look backwards.

THE TWO THAT LOOK LIKE THEY NEED JAVASCRIPT

VARIANT	GENERATES	USE IT FOR
<code>group-hover:</code>	<code>:is(:where(.group):hover *)</code>	parent hovered, child styled. Put <code>group</code> on the parent
<code>peer-checked:</code>	<code>:is(:where(.peer):checked ~ *)</code>	a control styling a later sibling. Put <code>peer</code> on the control

READ THEM OUT LOUD

“an element inside a hovered `.group`”, and “a later sibling of a checked `.peer`”. A combinator and a pseudo-class.

THE CONTROL MUST COME FIRST

`~` is the general sibling combinator. **CSS cannot look backwards**, so a `peer` after the thing it styles does nothing at all.

WORKED EXAMPLE

ONE LINK, FOUR STATES, IN THE BROWSER

- 1 A plain nav link. Add `hover:` . Check it.
- 2 Add `focus-visible:` . Check it **with Tab**, not the mouse.
- 3 Add `dark:` . Flip the OS setting and watch it change.
- 4 Now delete the focus one and Tab again. Put it back.

Step 4 is the point. Ten seconds without a focus ring is worth more than a slide about accessibility.

BOTH RAMPS, ON SCREEN AT ONCE

the same card, two ramps, contrast computed live

reload

open

LIGHT RAMP

GATEWAY

Certificate expires in 6 days

Renew it before Friday or the gateway starts refusing connections.

Renew now

DARK RAMP

GATEWAY

Certificate expires in 6 days

Renew it before Friday or the gateway starts refusing connections.

Renew now

every colour in both cards, resolved from the rendered page – and what week 3 called it

@theme token	week 3 (tokens.css)	light	dark
--color-page	--color-bg	☐ rgb(255, 255, 255)	☒ rgb(16, 20, 30)
--color-surface	--color-surface	☐ rgb(248, 249, 252)	☒ rgb(25, 30, 42)
--color-edge	--color-border	☐ rgb(199, 205, 219)	☒ rgb(63, 71, 90)
--color-ink	--color-text	☒ rgb(15, 23, 41)	☐ rgb(242, 245, 248)
--color-text-muted	--color-text-muted	☒ rgb(82, 88, 104)	☐ rgb(182, 183, 185)

► **HANDS-ON** demos/07-dark/ — ask yourself “**WHAT CHANGED?**” Give it fifteen seconds.

THE HONEST COSTS — AND I WROTE THESE BEFORE TAILWIND

CLASS BLOAT

“Many classes in HTML can make the code messy.”

OVER-RELIANCE

“Over-reliance can limit learning of native CSS.”

SEPARATION

“You want clean separation between structure and styling.”

— ABOUT BOOTSTRAP. EVERY ONE OF THEM APPLIES HERE.

WHAT YOU WILL SEE IN OLDER TUTORIALS

TAILWIND V3 — YOU WILL MEET THIS EVERYWHERE

```
<script src="https://cdn.tailwindcss.com"></script>
<script>
  tailwind.config = {
    theme: {
      extend: {
        colors: { brand: '#182df2' },
      },
    },
  };
</script>
```

V4 — WHAT WE USE

```
<script src="vendor/tailwind.js"></script>
<style type="text/tailwindcss">
  @theme {
    --color-brand-500: hsl(234 89% 52%);
  }
</style>
```

Same idea, and the class names barely moved. If a tutorial shows the left one, the classes still apply — only the config does not. **Measured** on this course's v4 file: the config line throws `tailwind is not defined`, `bg-brand-600` gets no rule, and white text sits on nothing — 1.00:1. With no network, the CDN line loads nothing at all.

THE LADDER HAS THREE RUNGS, AND YOU ARE ON THE SECOND

SIX COPIES

Today, if you do nothing. Two of them have already drifted.

ONE DEFINITION, COPIED

`components.html`. One place a component is defined, one place to change it. **This is the assignment, and it is graded by measurement.**

ONE FUNCTION

Week 10. Then copying stops being a step anybody can forget — and markup can be produced from **data**.

```
const BUTTON = 'cursor-pointer rounded-lg bg-brand-500 px-5 py-2 font-semibold text-white';  
const button = (label) => `
```

**Read that. Do not write it — it needs
template literals, which is week 8.**

AND THE ONE PLACE IT IS RIGHT

WHEN YOU DO NOT OWN THE MARKUP

A base rule for every `<a>` in prose. A `::selection` colour. A third-party widget you cannot put a class on.

A HANDFUL OF RULES, IN A BASE LAYER

That is the whole legitimate use. If you want it everywhere, what you want is a **component**.

Which is why `@apply` is forbidden in the assignment: it would let you avoid the thing the assignment is about.

REPEATED MARKUP — THE THREE RUNGS

RUNG	WHAT IT IS	WHAT IT COSTS
copies	the same class list typed out N times	they drift within a fortnight, silently
a gallery page	defined once, copied from there	a discipline, so it can be broken. This is the assignment, and cycle 5
a function	one string, N calls	nothing. Week 10 , and it needs JavaScript
<code>@apply</code>	not a rung. Moves styles into a stylesheet	every cost of having a stylesheet, back

@apply's one legitimate use: elements whose markup you do not control — prose links, `::selection`, a third-party widget.

THE FIVE, AND HOW TO CATCH EACH IN YOUR OWN FILE

1. "IT'S INLINE STYLES"

DevTools → Rendering → emulate

`prefers-color-scheme: dark` ·

A `dark:` · class answers; a

`style=""` cannot.

2. MEMORISING THE CLASS NAMES

Hover the class in VS Code: the

Tailwind extension the repo

recommends shows the CSS it generates.

3. @apply EVERYWHERE

Search your CSS for `@apply`. More than the one legitimate base-layer use hands back the stylesheet you were leaving.

4. IGNORING THE SCALE

Search your HTML for `-[` ·. Every

arbitrary value needs a reason you can

say; `p-[13px]` · beside `p-3` has

none.

5. CLASS SOUP WITH NO COMPONENT

Search for your button's full class

string. More than one exact hit is a

copy waiting to drift.

THE FIVE, ONE LINE EACH

1. "IT'S INLINE STYLES"

A class carries conditions — hover, width, scheme. An attribute carries a value, at `1,0,0,0`.

2. MEMORISING THE NAMES

Know the CSS underneath. Ask [demos/03-translator/](#) for the rest.

3. @apply EVERYWHERE

Solves the wrong half. Six `class="btn"` is still six copies.

4. IGNORING THE SCALE

Rhythm is a property of the set. One off-scale box is invisible; three are not.

5. NO COMPONENT

The copy that loses a class looks perfect until somebody touches it.

WHERE TO LOOK, IN ORDER

- 1 **demos/03-translator/**
— paste CSS, get the class. It measures.
- 2 **Tailwind CSS IntelliSense**
— and it flags a class that does not exist.
- 3 **tailwindcss.com/docs**
— the search takes CSS property names.
- 4 **DevTools**
— the styles panel. It is ordinary CSS and always was.

Starting at the docs is memorising.
Starting at the translator is translating.